AICTE VERIFIED STUDENT REGISTER

System Architecture and Workflow

Component boundaries, data flow, key sequences and the limits of the prototype architecture.

Document: 05-ARCH Version: 1.0 Date: 24 October 2026

Audience: AI coding agent + project owner

Document-control note
This is an implementation specification for a synthetic-data prototype, not an official AICTE standard, production approval, legal opinion, or proof of live government integration.

Source discipline: requirements from the PS 26263 implementation report and Sigma Batch curriculum are distinguished from proposed engineering choices. Any item that depends on a real policy, approval, API, credential trust chain, or deployment environment must be confirmed rather than guessed.

1. Architecture overview

The architecture intentionally uses a modular Express application with separate services/modules, not a fleet of microservices. It follows the revised MERN stack and omits the original report's PostgreSQL, Redis/BullMQ, Keycloak, Python ML and Kubernetes components from the first prototype.

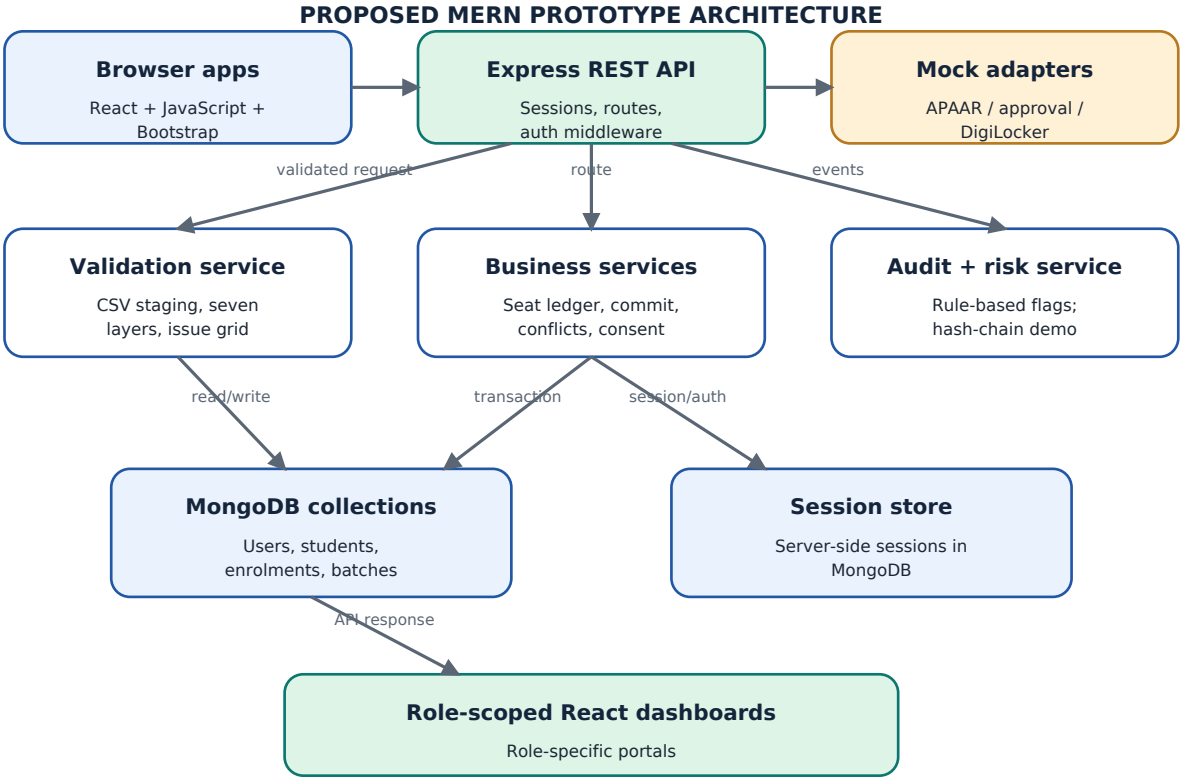

Diagram note: this is a logical view, not a claim that the implementation has already been built. Arrows show major responsibilities, not every individual HTTP call.

2. Component responsibilities

Component	Owns	Must not do
React client	Screens, field-level feedback, navigation, presentation of backend state.	Make authoritative seat decisions, trust client role data, or store secrets.
Express routes/controllers	HTTP parsing, input shape validation, invoking service methods and consistent responses.	Contain all domain logic in route handlers or bypass authorization.
Authentication middleware	Resolve session/user, check account status and required role/scope.	Trust an arbitrary role or institution ID from the request body.
Batch/ingestion service	Accept file, create batch, parse CSV safely, write staged records and statuses.	Insert incoming rows directly into the official register.
Validation service	Run deterministic L1-L6 checks and record L7 risk signals.	Treat risk flag as fraud proof or call real external APIs without access.
Seat-ledger service	Reserve/release capacity consistently with enrolment mutations.	Perform an unprotected read-then-write race-prone count.
Registry/commit service	Apply certified records and version/history/audit effects.	Partially commit records if seat reservation fails.
Conflict service	Detect and resolve potential cross-college active-enrolment conflicts.	Auto-accuse or delete the record based on a match alone.
Consent/verification service	Enforce requested fields, purpose, expiry and student approval.	Return private student data before an authorised basis exists.
Mock adapters	Simulate external data contracts in tests/demo.	Claim live integration, real API success or real identity verification.
MongoDB	Persist users, students, enrolments, batches, capacity, conflicts, consent and audit.	Be accessed directly from the frontend.

3. Upload and commit sequence

1. React submits a CSV file to POST /api/batches after client-side pre-check.

2. Express authenticates the operator and derives the institution scope from the session, not from a trusted client-supplied field.

3. Ingestion service validates file limits, creates a batch and parses into staged row records.

4. Validation service writes row-specific issues with severity and rule codes. A mock external adapter may return verified, mismatch or pending demo outcomes.

5. Operator fixes allowed fields and asks to revalidate changed rows.

6. The College Head reviews the batch and certifies only after blocking errors are cleared and required warning dispositions are present.

7. Commit service re-checks permissions, batch state, policy version and capacities inside the commit operation. It updates enrolments and seat counters consistently and records history and audit entry.

8. If any required update fails, the transaction rolls back. The batch remains uncommitted with an actionable error state.

9. Dashboard reads updated data. Risk scoring and notifications are later tasks; a prototype can trigger rule evaluation after commit without promising a durable queue.

4. Conflict resolution sequence

Actor/system	Action	Guardrail
Conflict detector	Find apparent active enrolments for same student across colleges/academic year.	Candidate match only; not a fraud conclusion.
Institution A/B users	Review their own enrolment side and provide allowed state updates/evidence.	Only see permitted minimum details about the other institution.
Conflict service	Validate transition and release/reserve seat as appropriate.	Do not close the case if required resolution conditions are missing.
Audit service	Record state change, reason, actor and timestamp.	Preserve historical events; no silent replacement of history.
AICTE officer	Review/escalate unresolved case.	Escalation does not itself determine guilt.

5. Consent and QR sequence

1. Verifier creates a request with student reference, purpose, requested fields and expiration.
2. Student portal shows the complete request and allows approve or deny.
3. Server stores the decision and checks consent/expiry on every result retrieval.
4. Verifier receives only fields that were approved.
5. Credential/QR demo points to a verification reference. The verifier checks the stored demo status.
6. Revoke updates status; an altered/malformed reference returns a generic invalid result. Do not label an unsigned QR as signed or cryptographically authenticated.

6. Synchronous versus background processing

For the first prototype, keep the data volume bounded and document the behaviour if the server restarts during validation. If a lightweight in-process worker is used, record status in MongoDB and mark unfinished jobs retry-required on startup. A durable queue (for example Redis/BullMQ) is a later scale upgrade, not part of the required stack. Do not claim reliable large-scale background processing without durable job recovery and load testing.

7. Deployment shape

- Development: React dev server, Express API, and a supported MongoDB local replica-set or test deployment when transaction flows are enabled.
- Keep frontend and backend origins explicit; configure CORS and cookie behaviour only if they are cross-origin.
- Prototype deployment can be a conventional Node frontend/API hosting setup plus MongoDB hosting; exact provider is TBD until chosen and tested.
- Production architecture in the original report (Kubernetes, advanced key management, HSM, multi-region DR and government cloud) is not delivered by this MVP.

8. Observability minimum

- Log request ID, route, status code, duration and actor reference where safe.
- Never log passwords, session IDs, tokens, full student IDs, full DOB or raw consent payloads.
- For failed commit, record batch ID and safe error code; do not print secret values.
- Expose health endpoints for the app and database connection status.
- Keep enough telemetry to reproduce a test failure and report actual processing times.